

HANDS - ON: FUNDAMENTOS DE LLMS CON MODELOS LLAMA

Una vez vista la masterclass **Fundamentos de LLMs y Arquitectura de Llama**, se proporciona el siguiente **Colab** para construir, en vivo, la primera llamada a Llama y medir su comportamiento.

Usamos **Groq** para tener acceso a inferencia de Llama con GPU gratuita, sin necesidad de infraestructura propia.

De igual manera, se proporciona la **solución** de este notebook a través del siguiente [enlace](#).

CONFIGURACIÓN DEL ENTORNO

Antes de llamar a Llama necesitamos activar la GPU gratuita de Colab y conectar nuestra cuenta de **Groq**. La librería `groq` es el cliente oficial en Python para hacer llamadas a la API.

COLAB SECRETS

Para no exponer tu **API key** directamente en el código, Colab ofrece un panel de **Secrets** (ícono de llave en la barra lateral izquierda) donde puedes guardarla de forma segura, añadiendo un nombre asociado a la **API key** para guardarla dentro de una variable y usarla dentro del notebook.

```
# Instalar cliente de Groq y leer API key desde Colab Secrets
!pip install Groq
```

```
Collecting Groq
  Downloading groq-1.7.0-py3-none-any.whl.metadata (16 kB)
Requirement already satisfied: anyio<5,>=3.5.0 in /usr/local/lib/python3.13/dist
Requirement already satisfied: distro<2,>=1.7.0 in /usr/local/lib/python3.13/dis
Requirement already satisfied: httpx<1,>=0.23.0 in /usr/local/lib/python3.13/dis
Requirement already satisfied: pydantic<3,>=1.9.0 in /usr/local/lib/python3.13/d
Requirement already satisfied: sniffio in /usr/local/lib/python3.13/dist-package
Requirement already satisfied: typing-extensions<5,>=4.14 in /usr/local/lib/pyth
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.13/dist-packa
```

```
Requirement already satisfied: certifi in /usr/local/lib/python3.13/dist-package
Requirement already satisfied: httpcore==1.* in /usr/local/lib/python3.13/dist-p
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.13/dist-packa
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/lib/python3.
Requirement already satisfied: pydantic-core==2.46.5 in /usr/local/lib/python3.1
Requirement already satisfied: typing-inspection>=0.4.2 in /usr/local/lib/python
Downloading groq-1.7.0-py3-none-any.whl (143 kB)
```

143.8/143.8 kB 7.1 MB/s eta 0:00:00

Installing collected packages: Groq

Successfully installed Groq-1.7.0

✓ DE PALABRAS A TOKENS

Antes de que Llama genere una sola palabra, tu prompt se divide en **tokens**. Vamos a definir un prompt de ejemplo que después enviaremos a Llama.

```
# Definir un prompt de ejemplo (una pregunta real de tu propio contexto)
from google.colab import userdata
from groq import Groq

api_key = userdata.get("GROQ_API_KEY")
client = Groq(api_key = api_key)

print("API key cargada correctamente")

pregunta = "¿Qué es un PLC y para qué se utiliza en la automatización industria

API key cargada correctamente
```

✓ PRIMERA LLAMADA A LLAMA (ZERO-SHOT)

Un LLM predice el siguiente token más probable, **no busca la respuesta en una base de datos**. Vamos a enviar el prompt a Llama en modo zero-shot (sin ejemplos previos) y ver la respuesta generada.

```
# Enviar el prompt a Llama en Groq y mostrar la respuesta generada
respuesta = client.chat.completions.create(
    model="openai/gpt-oss-20b",
    messages=[
        {"role": "user", "content": pregunta}
    ]
)

print(respuesta.choices[0].message.content)
```

- Líneas de producción (encadenamiento de operaciones).
 - Operaciones de carga/descarga, mezclado, embotellado, empaquetado, etc.
2. ****Regulación de variables (control de velocidad, presión, nivel, temperatur**
 - PID (Proporcional-Integral-Derivativo) implementado en el PLC para manten
 3. ****Detección y gestión de fallos****
 - Supervisión de sensores y actuadores.
 - Generación de alarmas y log de eventos.
 4. ****Comunicación e integración con sistemas superiores****
 - Interfaz con SCADA/ HMI para visualizar el estado y ajustar parámetros.
 - Integración con ERP para trazabilidad y gestión de producción.
 5. ****Automatización de maquinaria específica****
 - Control de robots industriales, transportadores, prensas, tornos, fresado
 6. ****Implementación de lógica compleja de negocio****
 - Rutas de producto, asignación de tareas, optimización de tiempos de ciclo
 7. ****Gestión de seguridad y cumplimiento normativo****
 - Sistemas de parada de emergencia, cumplimiento con normas IEC 61508/IEC 6

Ventajas de usar PLC en la industria

Ventaja	Por qué es importante
----- -----	
Reprogramabilidad	Cambiar la lógica sin reemplazar hardware.
Fiabilidad	Diseñado para 24/7 sin fallos críticos.
Escalabilidad	Se añaden módulos de I/O para crecer la planta.
Diagnóstico	Registro de fallos, historial de eventos, mantenimiento pr
Compatibilidad	Estándares abiertos (IEC 61131-3), interoperabilidad co
Seguridad	Funciones integradas para paradas de emergencia y límites se

- Su flexibilidad, fiabilidad y compatibilidad lo convierten en la columna vertebral de la IA generativa.

✓ CUÁNTOS TOKENS CONSUMIÓ EL PROMPT

La respuesta de la API trae un resumen (`usage`) con el número de tokens de entrada y de salida. Es la forma más directa de responder: ¿cuántos tokens consumió mi prompt?

```
# Mostrar cuántos tokens tuvo el prompt y cuántos tuvo la respuesta
print("Tokens del prompt:", respuesta.usage.prompt_tokens)
print("Tokens de respuesta:", respuesta.usage.completion_tokens)
print("Tokens totales:", respuesta.usage.total_tokens)
```

```
Tokens del prompt: 86
Tokens de respuesta: 1005
Tokens totales: 1091
```

✓ MIDIENDO LA LATENCIA DE INFERENCIA

Elegir el tamaño correcto de modelo es una decisión de ingeniería, no solo de potencia bruta. Vamos a medir cuánto tarda Llama en responder el mismo prompt.

```
# Medir el tiempo de respuesta de Llama para el mismo prompt
import time

inicio = time.time()

respuesta = client.chat.completions.create(
    model = "openai/gpt-oss-20b",
    messages = [
        {"role": "user", "content": pregunta}
    ]
)

fin = time.time()

tiempo_respuesta = fin - inicio

print("Tiempo de respuesta:", tiempo_respuesta, "segundos")
print(respuesta.choices[0].message.content)
```

4. Ventajas de los PLC

1. **Robustez y fiabilidad** - Diseñados para operar sin interrupciones en ent
2. **Escalabilidad** - Se pueden añadir módulos de I/O según la necesidad del
3. **Facilidad de mantenimiento** - Cambio de cables sin necesidad de desmonta
4. **Seguridad** - Programas de parada de emergencia, protección contra sobre-
5. **Integración** - Comunicación con sistemas SCADA/HMI y ERP para supervisió

5. Ciclo de trabajo típico

1. **Lectura de entradas** - El PLC lee la señal de cada sensor.
2. **Procesamiento** - Ejecuta la lógica programada (ladder, bloques, etc.).
3. **Generación de salidas** - Activa actuadores según el resultado de la lógi
4. **Descarga de programa** - En caso de actualización, el PLC puede recibir u

Este ciclo se repite una y otra vez en un intervalo que puede ser desde micros

6. Programación

Los lenguajes más comunes:

- **Ladder Diagram (LD)** - parecido a un diagrama de relés, muy visual.
- **Function Block Diagram (FBD)** - bloques funcionales conectados.
- **Structured Text (ST)** - similar a Pascal/ C, útil para algoritmos complej
- **Instruction List (IL)** - lenguaje ensamblado, menos usado hoy.
- **Sequential Function Charts (SFC)** - secuenciación de pasos, ideal para pr

El programador crea una “tarea” (task) que se ejecuta en cada ciclo. La lógica

7. Ejemplo práctico rápido

Supongamos una línea de embotellado que debe:

1. Detectar que una botella llega al punto de llenado.
2. Activar la válvula de llenado durante 5 segundos.
3. Desactivar la válvula y mover la botella al siguiente paso.
4. Detener la línea si el sensor de nivel indica que la botella está vacía.

El PLC leerá el sensor de posición (Entrada), activará la válvula (Salida) con

8. Conclusión

Un PLC es la “nervadura” de la automatización industrial moderna. Su capacidad

✓ COMPARANDO DOS TAMAÑOS DE MODELO

Repite la misma llamada usando la versión más grande de Llama disponible en Groq y compara tiempo de respuesta y calidad contra el modelo ligero.

```
# Repetir la llamada con un modelo Llama más grande y comparar t
import time

inicio = time.time()

respuesta_grande = client.chat.completions.create(
    model = "openai/gpt-oss-120b",
    messages=[
        {"role": "user", "content": pregunta}
    ]
)

fin = time.time()

tiempo_grande = fin - inicio

print("Tiempo de respuesta del modelo grande:", tiempo_grande, "
print("\nRespuesta del modelo grande")
print(respuesta_grande.choices[0].message.content)

print("MODELO LIGERO")
print("Tiempo:", tiempo_respuesta, "segundos")
print("Respuesta:")
print(respuesta.choices[0].message.content)

print("\n" + "=" * 60 + "\n")

print("MODELO GRANDE")
print("Tiempo:", tiempo_grande, "segundos")
print("Respuesta:")
print(respuesta_grande.choices[0].message.content)
```

3. ****Tiempo de respuesta rápido**** - Ciclos de escaneo de decenas a cientos de
4. ****Facilidad de mantenimiento**** - Diagnósticos integrados y módulos intercam
5. ****Escalabilidad**** - Se pueden ampliar entradas/salidas y protocolos sin ree
6. ****Estandarización**** - Programas compatibles con normas internacionales (IEC

3. Ejemplo práctico simplificado (Ladder)

Supongamos una cinta transportadora que debe arrancar cuando se pulsa un ****bot**

```
|---[ ]---[ ]---[/]---( )---|
|   I1   I2   I3   Q1   |
| Inicio Stop Límite Motor|
|---
```

- ****[]**** = contacto normalmente abierto (NO) (botón de inicio `I1`).
- ****[/]**** = contacto normalmente cerrado (NC) (botón de parada `I2`).
- ****[]**** = sensor de límite `I3` (NC para que el motor se apague al activarse
- **** () **** = bobina (salida) que energiza el motor `Q1`.

Cuando el operador pulsa ***Inicio*** (`I1=1`) y mientras ***Parada*** y ***Límite*** está

4. Tendencias actuales

Tendencia	Impacto en PLCs
PLC híbrido (con CPU y FPGA)	Mayor velocidad de procesamiento y capaci
Edge Computing	PLCs integran funciones de pre-procesamiento de datos y
Integración con IoT	Puertos Ethernet con protocolos MQTT, OPC UA y API
Software de programación basado en la web	Herramientas de ingeniería q
Seguridad cibernética	Incorporación de firewalls, autenticación de usu

5. Resumen rápido

- ****Qué es:**** Un PLC es un controlador digital, programable y robusto, pensado
- ****Para qué sirve:**** Ejecuta lógicas de control que interconectan sensores y
- ****Por qué se elige:**** Fiable, flexible, fácil de mantener, escalable y compa

En definitiva, el PLC es el corazón de la automatización moderna. proporcionan

✓ CHALLENGE: COMPARADOR DE MODELOS LLAMA

Una vez visto el ***Hands-On: Fundamentos de LLMs***, se presenta el siguiente reto para que el alumnado pueda repasar y reforzar lo aprendido dentro de la clase.

Se construirá un pequeño **comparador que envíe 3 preguntas** reales a Llama y registre, para cada una, el **tiempo de respuesta** y el **número de tokens** consumidos.

IMPORTANTE: Para su revisión, **es indispensable que los apartados anteriores se encuentren llenados con el código visto durante la sesión.**

✓ INSTRUCCIONES:

1. Carga la API key y genera 3 preguntas:

- Lee la API key de llama ya configurada desde **Colab Secrets**.
- Construye una lista vacía llamada `preguntas` y agrégle 3 preguntas frecuentes de tu propio contexto (soporte técnico, tienda, escuela, etc.).

```
# Leer API key desde Colab Secrets
# Leer API key desde Colab Secrets

from google.colab import userdata
from groq import Groq

api_key = userdata.get("GROQ_API_KEY")
client = Groq(api_key=api_key)
```

```
# Definir la lista de preguntas

preguntas = []

preguntas.append("¿Qué es un PLC y para qué se utiliza en la automatización inc
preguntas.append("¿Qué es una prueba de confiabilidad en un producto electrónic
preguntas.append("¿Qué es Python y para qué se utiliza en inteligencia artifici
print(preguntas)
```

```
['¿Qué es un PLC y para qué se utiliza en la automatización industrial?', '¿Qué
```

2. Consulta la primera pregunta: Envía la primera pregunta a Llama usando el modelo más ligero disponible en Groq. Guarda la respuesta, el tiempo de respuesta y el número de tokens en un diccionario llamado `resultado_1`.

```
# Consultar la primera pregunta y guardar el resultado en resultado_1
import time

inicio = time.time()
```

```

respuesta_1 = client.chat.completions.create(
    model="openai/gpt-oss-20b",
    messages=[
        {"role": "user", "content": preguntas[0]}
    ]
)

```

```
fin = time.time()
```

```

resultado_1 = {
    "pregunta": preguntas[0],
    "respuesta": respuesta_1.choices[0].message.content,
    "tiempo_respuesta": fin - inicio,
    "tokens": respuesta_1.usage.total_tokens
}

```

```
print(resultado_1)
```

```
{'pregunta': '¿Qué es un PLC y para qué se utiliza en la automatización industri
```

3. Repite para la segunda y tercera pregunta: Crea `resultado_2` y `resultado_3` de la misma forma.

```

# Consultar la segunda pregunta y guardar el resultado en resultado_2
inicio = time.time()

```

```

respuesta_2 = client.chat.completions.create(
    model="openai/gpt-oss-20b",
    messages=[
        {"role": "user", "content": preguntas[1]}
    ]
)

```

```
fin = time.time()
```

```

resultado_2 = {
    "pregunta": preguntas[1],
    "respuesta": respuesta_2.choices[0].message.content,
    "tiempo_respuesta": fin - inicio,
    "tokens": respuesta_2.usage.total_tokens
}

```

```
print(resultado_2)
```

```
{'pregunta': '¿Qué es una prueba de confiabilidad en un producto electrónico?',
```

```
# Consultar la tercera pregunta y guardar el resultado en resultado_3
inicio = time.time()

respuesta_3 = client.chat.completions.create(
    model="openai/gpt-oss-20b",
    messages=[
        {"role": "user", "content": preguntas[2]}
    ]
)

fin = time.time()

resultado_3 = {
    "pregunta": preguntas[2],
    "respuesta": respuesta_3.choices[0].message.content,
    "tiempo_respuesta": fin - inicio,
    "tokens": respuesta_3.usage.total_tokens
}

print(resultado_3)

{'pregunta': '¿Qué es Python y para qué se utiliza en inteligencia artificial?',
```

4. Junta los resultados: Agrega `resultado_1`, `resultado_2` y `resultado_3` a una lista vacía llamada `resultados`.

```
# Definir la lista resultados y agregar los tres diccionarios
resultados = []

resultados.append(resultado_1)
resultados.append(resultado_2)
resultados.append(resultado_3)

print(resultados)

[{'pregunta': '¿Qué es un PLC y para qué se utiliza en la automatización industr
```

5. Muestra la comparación: Imprime `resultados` y concluye si el modelo ligero resolvió las 3 preguntas satisfactoriamente.

```
# Mostrar la tabla final de resultados
print("RESULTADOS DEL COMPARADOR")
print("=" * 60)

for i, resultado in enumerate(resultados, start=1):
    print(f"\nPregunta {i}:")
    print(resultado["pregunta"])
```

```

print("\nRespuesta:")
print(resultado["respuesta"])
print(f"\nTiempo de respuesta: {resultado['tiempo_respuesta']}")
print(f"Tokens consumidos: {resultado['tokens']}")
print("-" * 60)

print("\nCONCLUSIÓN")
print("=" * 60)
print("El modelo ligero resolvió satisfactoriamente las 3 preguntas")

```

RESULTADOS DEL COMPARADOR

=====

Pregunta 1:

¿Qué es un PLC y para qué se utiliza en la automatización industrial?

Respuesta:

****PLC**** (Programmable Logic Controller, en español ***Controlador Lógico Programable***)

¿Qué es un PLC?

Característica	Descripción
Hardware robusto	Construido para operar en entornos industriales (temperaturas extremas, vibración, humedad)
Entradas/Salidas (I/O)	Conecta sensores, actuadores, motores, válvulas
Memoria	Almacena el programa de control, datos de operación, parámetros
Procesador	Ejecuta el programa con determinismo (tiempo de ciclo fijo)
Comunicación	Posee interfaces estándar (PROFIBUS, EtherNet/IP, Modbus, RS-485)
Programación	Se programa en lenguajes definidos por la norma IEC 61131 (Ladder Logic, Structured Text, Function Block Diagram)

¿Para qué se utiliza?

- **Control de maquinaria y líneas de producción****
 - Encender/apagar motores, controlar velocidad con variadores, sincronizar
- **Automatización de procesos****
 - Registros de temperatura, presión, nivel, flujo. Control de procesos químicos
- **Seguridad y protección****
 - Supervisión de dispositivos de parada de emergencia, alarmas, barreras de seguridad
- **Interfaz con sistemas de supervisión****
 - Recolección de datos en tiempo real para sistemas SCADA/MES, generación de reportes
- **Adaptabilidad y escalabilidad****
 - Se puede ampliar añadiendo módulos I/O, módulos de comunicación, módulos de procesamiento

Ventajas clave

Ventaja	Por qué importa
Fiabilidad	Diseñado para operar continuamente sin fallos.
Determinismo	Tiempo de respuesta predecible, crucial en procesos críticos
Flexibilidad	Cambios de configuración mediante reprogramación, sin necesidad
Escalabilidad	Añadir módulos I/O o de comunicación sin rehacer todo el sistema
Seguridad	Certificaciones IEC 61508, SIL 1-3, IEC 61511 (para procesos críticos)
Costo-efectividad	Menor tiempo de puesta en marcha y mantenimiento comparado con PLC

Ejemplo práctico